



United International University (UIU)
Digital Signal Processing laboratory
Course Code EEE 3310

Lab Sheet 3

Discrete Time System and Response

Prerequisite

Before starting this section, students should know:

1. Concepts of MATLAB
2. Contents of Lab 2

Lab Outcome

After finishing this lab students should be able to:

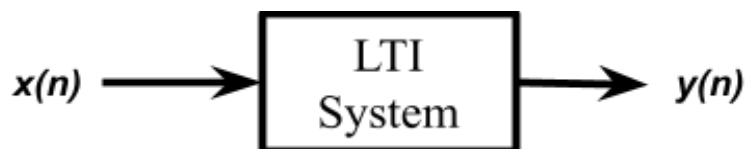
1. Solve the difference equation using filter function
2. Determine system response using filter function
3. Convolution and correlation and their practical use

Outlines of this Lab

1. Lab Session 3.1: Difference equation
2. Lab Session 3.2: Convolution and Correlation of sequences
3. Lab Session 3.3: Sequence Correlation
4. Lab Session 3.4: Correlation Coefficient

What is a Discrete-Time System:

A discrete-time system is a mathematical algorithm or physical device that processes an input discrete-time signal to generate an output discrete-time signal. In digital signal processing, these systems perform operations such as filtering, delaying, or scaling on sampled data.



A LTI system is represented by either Impulse response or a difference equation.

3.1 Difference equation

A difference equation represent the output-input relationship between $x(n)$ & $y(n)$ with constant coefficient finite order and difference equation.

$$\sum_{k=0}^N a_k y(n-k) = \sum_{m=0}^M b_m x(n-m) \dots \dots \dots (i)$$

here, $\sum_{k=0}^N a_k y(n-k)$ is the **Recursive** part and $\sum_{m=0}^M b_m x(n-m)$ is the **Non-Recursive** part.

“n” is the order of inputs and “m” is the order of outputs.

A function called **filter** is available in MATLAB to solve difference equation numerically, given the input and the difference equation coefficients. In its simplest form, this equation is called by,

$$y = \text{filter}(b, a, x)$$

$$\text{Where, } b = [b_0, b_1, \dots, b_M]$$

$$\text{And, } a = [a_0, a_1, \dots, a_N]$$

here, length(y)=length(x)

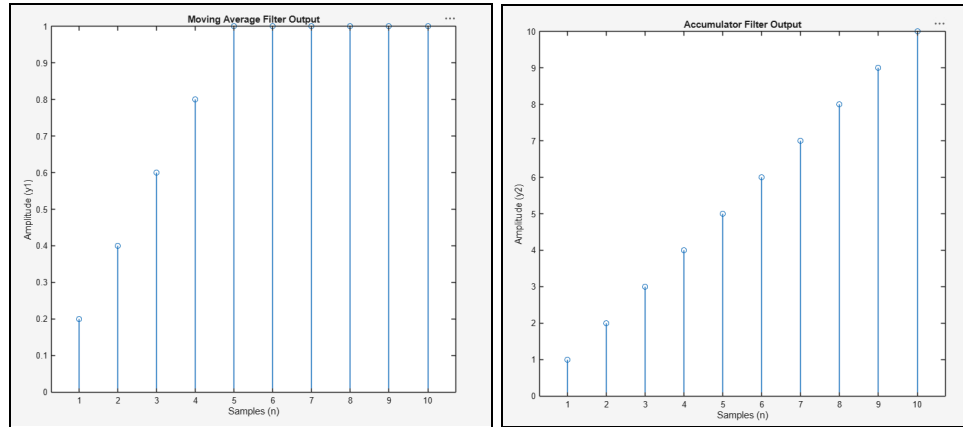
It is to be noted that unlike the conv command the filter command includes only the initial transient but not final transient. In a general setting the general form of the difference equation given in (1) is also known as the ARMA model. The recursive section related to y(n) is known as the auto-regressive part and the non-recursive part is known as the moving average.

Example 3.1: Moving Average Filter

MATLAB Code:

```
%% Use of filter command
clear all
close all
x= ones(1,10);
% Moving average:
L=5
b1= (1/L)*ones(1,L);
a1=1;
y1= filter(b1,a1,x);
figure
stem(y1)
title('Moving Average Filter Output');
xlabel('Samples (n)');
ylabel('Amplitude (y1)');
% Accumulator:
b2=1;
a2=[1 -1];
y2= filter(b2,a2,x);
figure
stem(y2)
title('Accumulator Filter Output');
xlabel('Samples (n)');
ylabel('Amplitude (y2)');
```

Output:



3.2 Convolution and Correlation of sequences

Mathematically, a linear convolution sum is defined as

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)h(n - k) \dots \dots \dots (ii)$$

In general, the convolution operation is used to describe the response of an LTI system. In DSP it is an important operation and has many other uses. MATLAB does provide a built-in function called `conv` that computes the convolution between two finite-duration sequences.

Description:

`y = conv(h, x)` convolves vectors `u` and `v`. Algebraically, convolution is the same operation as multiplying the polynomials whose coefficients are the elements of `h` and `x`.

Thus the `conv` function assumes that the two sequences begin at `n = 0` and is invoked by

`y = conv(x, h)`

here, $L_y = L_h + L_x - 1$

Example 3.2 : Convolution

MATLAB Code:

```
x=[3 11 7 6];
h=[4 -3 2];
y = conv(x,h);
disp('y=');
disp(y);
```

Output: `y = 12      35      1      25      -4      12`

Modified function for convolution:

However, the `conv` function neither provides nor accepts any timing information if the sequences have arbitrary zero position. To solve this inconvenience, a simple extension of the `conv` function, called `conv_m`, which performs the convolution of arbitrary support sequences can now be designed as

MATLAB Code:

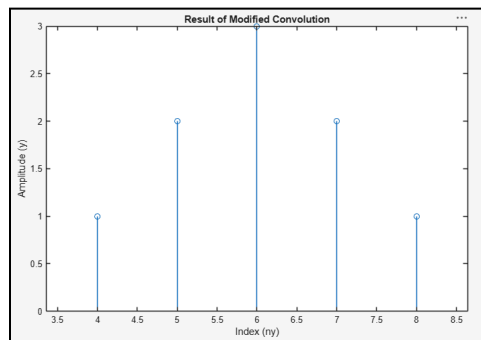
```
function [y, ny] = conv_m(x, nx, h, nh)
% Modified convolution routine for signal processing
% -----
% [y, ny] = convolution result
% [x, nx] = first signal
% [h, nh] = second signal
nyb = nx(1) + nh(1);
nye = nx(length(x)) + nh(length(h));
ny = [nyb:nye];
y = conv(x, h);
end
```

Example 3.3: Convolution using conv_m

MATLAB Code:

```
%% Test convolution:
x=[1 1 1];
nx=-1:1;
h=[1 1 1];
nh=5:7;
[y, ny] = conv_m(x, nx, h, nh);
figure
stem(ny,y)
```

Output:



3.3 Sequence Correlation

In the field of digital signal processing, correlation serves as a fundamental operation for assessing the resemblance between two discrete sequences. For two real-valued sequences of finite energy, $x(n)$ and $y(n)$, the **crosscorrelation** is represented by the sequence $r_{xy}(l)$ and is defined as:

$$r_{xy}(l) = \sum_{n=-\infty}^{\infty} x(n)y(n-l) \dots\dots\dots(iii)$$

In this context, the parameter l denotes the lag or shift. A specific instance where $y(n) = x(n)$ is known as **autocorrelation**, formulated as:

$$r_{xx}(l) = \sum_{n=-\infty}^{\infty} x(n)y(n-l) \dots\dots\dots(\text{iv})$$

A comparison between convolution and crosscorrelation reveals a significant mathematical similarity. Specifically, the crosscorrelation $r_{xy}(l)$ can be restructured as:

$$r_{xy}(l) = x(l) * y(-l) \dots\dots\dots(\text{v})$$

Likewise, the autocorrelation sequence $r_{xx}(l)$ can be expressed by:

$$r_{xx}(l) = \sum_{n=-\infty}^{\infty} x(l) * x(-l) \dots\dots\dots(\text{vi})$$

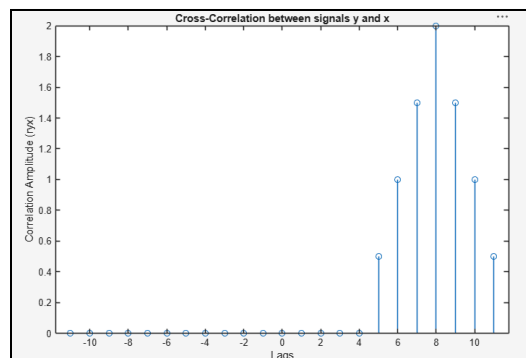
Consequently, for finite-duration signals, these measures can be effectively determined by utilizing the MATLAB **conv** routine.

Example 3.4: Correlation

MATLAB Code:

```
% Correlation
clear all
close all
x=[1 1 1 1 0 0 0 0 0 0 0];
y=[0 0 0 0 0 0 0 0.5 0.5 0.5 0.5];
Ts= 0.000001;
c= 3e8; % speed of light
[ryx lags] = xcorr(y,x);
figure
stem(lags,ryx)
title('Cross-Correlation between signals y and x');
xlabel('Lags');
ylabel('Correlation Amplitude (ryx)');
RR= ryx(13:end);
[Max_RR,max_ind]= max(RR)
Round_trip_delay= max_ind*Ts;
distance= 0.5*Round_trip_delay*c
```

Output: Max_RR = 2.0000
max_ind = 8
distance = 1200



3.4 Correlation Coefficient

The correlation coefficient, symbolized by r , quantifies the intensity of a linear association between two distinct variables. This metric ranges between -1 and +1, where a positive figure signifies a direct relationship between the sequences.

These coefficients are utilized to determine the degree of interdependence between data sets. The Pearson correlation coefficient for discrete-time sequences $x(n)$ and $y(n)$ is calculated using the following expression:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \dots\dots\dots \text{(vii)}$$

In this formula, $\bar{x}$ and $\bar{y}$ represent the arithmetic means of the sequences $x(n)$ and y , respectively.

Standard criteria for the interpretation of the resulting coefficient values are presented below:

Coefficient Range	Interpretation
0	Absence of linear relationship
+1	Perfectly positive linear association
-1	Perfectly negative linear association
0 to 0.3	Weak positive relationship
0 to -0.3	Weak negative relationship
0.3 to 0.7	Moderate positive relationship
-0.3 to -0.7	Moderate negative relationship
0.7 to 1	Strong positive relationship
-0.7 to -1	Strong negative relationship

Example 3.5: Correlation Coefficient

MATLAB Code:

```
%% Correlation coefficient calculation
close all
clear all
clc
X=[1 2 3];
%Y=-1.*[1 2 3];
Y=[3 2 1];
rxy= correlation_coef(X,Y)
```

Output: rxy = -1